



Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



1 REST APIs & HTTP Fundamentals

In this section:

- ✓ REST APIs
- ✓ HTTP Methods
- ✓ Status Codes
- ✓ Statelessness
- ✓ Idempotency
- ✓ Request-Response Flow

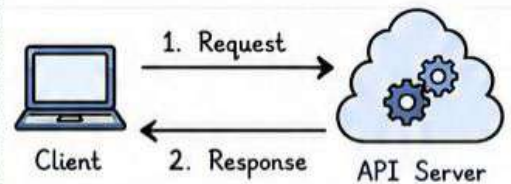
Quick Recap

REST (Representational State Transfer) is an architectural style for building APIs using HTTP.

Key Principles:

- Client-Server
- Stateless
- Cacheable
- Uniform Interface
- Layered System
- Code on Demand (optional)

Request → Response Lifecycle



Top 5 Questions in this Section

1 What is REST API? GET	Answer (Short) REST API is a way to build web services using HTTP methods. It allows communication between client and server.	Follow-up What are the 6 principles of REST? ?	Example / Use in Production Used in almost all modern web & mobile apps like Instagram, Amazon, Netflix for data communication.
2 What is the difference between PUT and PATCH? PUT vs PATCH	Answer (Short) PUT replaces the entire resource. PATCH updates only specific fields of the resource.	Follow-up Which one is idempotent? Why? ?	Example / Use in Production PUT /user/1 → replace all user data. PATCH /user/1 → update only email or name.
3 What is the difference between GET and POST? GET vs POST	Answer (Short) GET is used to retrieve data (safe & idempotent). POST is used to create new resource.	Follow-up Can we send data in GET request? ?	Example / Use in Production GET /users → fetch users POST /users → create a new user.
4 Why APIs should be stateless? 	Answer (Short) Stateless APIs don't store client state on server. Each request contains all necessary information.	Follow-up What are the benefits of stateless APIs? ?	Example / Use in Production Easier to scale, more reliable, fault tolerant and works well with load balancers.
5 What are HTTP Status Codes? 200	Answer (Short) HTTP Status Codes are 3-digit responses returned by server to indicate the result of a request.	Follow-up Name some 2xx, 4xx and 5xx codes. ?	Example / Use in Production 200 OK (success), 404 Not Found, 401 Unauthorized, 500 Internal Server Error.

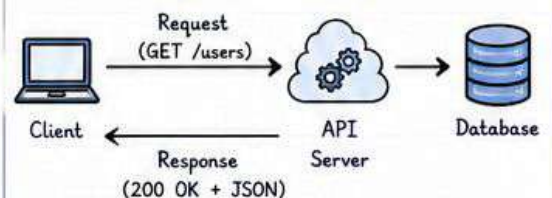
HTTP Methods Cheat Sheet

Method	Purpose	Idempotent
GET	Retrieve data	Yes ✓
POST	Create resource	No ✗
PUT	Replace resource	Yes ✓
PATCH	Update resource	No ✗
DELETE	Delete resource	Yes ✓

Common HTTP Status Codes

2xx Success	(200 OK, 201 Created)
3xx Redirection	(301 Moved, 304 Not Modified)
4xx Client Error	(400 Bad Request, 404 Not Found, 401 Unauthorized, 403 Forbidden)
5xx Server Error	(500 Internal Server Error, 502 Bad Gateway)

Request Flow Example





Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



2 Authentication & Authorization

Authentication Flow (JWT Example)

In this section:

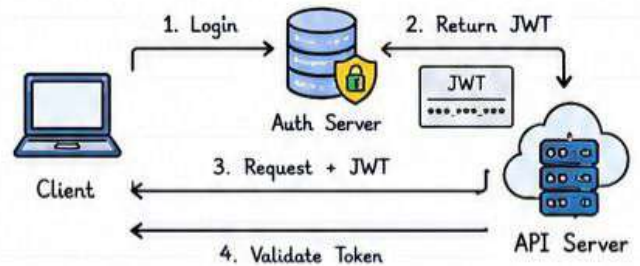
- ✓ Authentication vs Authorization
- ✓ Sessions & Cookies
- ✓ JWT (JSON Web Token)
- ✓ OAuth 2.0
- ✓ Access & Refresh Tokens
- ✓ CSRF Protection

Quick Recap

Authentication verifies who you are.
Authorization decides what you can do.

Common Methods:

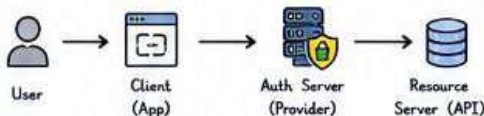
- Session Based Authentication
- Token Based Authentication (JWT)
- OAuth 2.0 for third-party access



Top 5 Questions in this Section

- | | | | | |
|---|--|---|--|--|
| 1 | What is the difference between Authentication and Authorization? | Answer (Short)
Authentication verifies identity (who you are). Authorization checks permissions (what you can do). | Follow-up
Can we have authorization without authentication? Why? | Example / Use in Production
Login (auth) → You are verified.
Accessing /admin (authz) → You must have admin role. |
| 2 | What is the difference between JWT and Session based auth? | Answer (Short)
JWT is stateless and stored on client. Session is stateful and stored on server. | Follow-up
Which one is more secure? Why? | Example / Use in Production
JWT for mobile apps & microservices.
Session for traditional web apps with server rendering. |
| 3 | What is Access Token and Refresh Token? | Answer (Short)
Access token is short-lived and used to access APIs. Refresh token is long-lived and used to get new access token. | Follow-up
Why not keep access token long-lived? | Example / Use in Production
Access token: 15 mins
Refresh token: 7 days
(Used to get new access token without login again) |
| 4 | How does OAuth 2.0 work? | Answer (Short)
OAuth 2.0 allows third-party apps to access user data without sharing credentials using authorization flow. | Follow-up
What are the main OAuth 2.0 flows? | Example / Use in Production
Google Login, GitHub Login, Facebook Login all use OAuth 2.0 Authorization Code Flow. |
| 5 | How can we prevent CSRF attacks? | Answer (Short)
Use CSRF tokens, SameSite cookies, and verify origin headers. | Follow-up
Is CSRF possible in JWT based auth? | Example / Use in Production
Yes, if JWT is stored in cookies. Use SameSite=Strict/None and CSRF token. |

OAuth 2.0 Authorization Code Flow

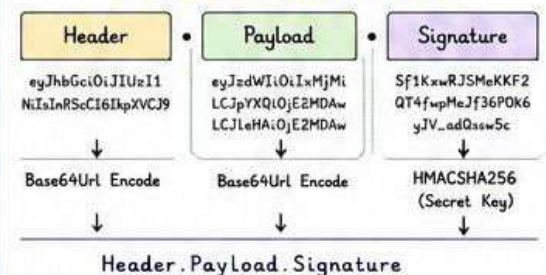


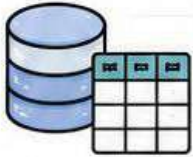
1. User initiates login
2. Client redirects to Auth Server
3. User authenticates
4. Auth Server returns authorization code
5. Client exchanges code for Access Token
6. Client accesses Resource Server with token

Token Storage Best Practices

- Store JWT in HttpOnly Cookies (prevents XSS attacks)
- Use Secure flag on cookies (HTTPS only)
- Set SameSite=Lax/Strict (prevents CSRF)
- Short-lived Access Token + Refresh Token rotation

Common Token Structure (JWT)





Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



3 SQL & Database Fundamentals

In this section:

- ✓ Tables & Keys
- ✓ Normalization
- ✓ Indexes
- ✓ Joins
- ✓ SQL Basics
- ✓ N+1 Query Problem

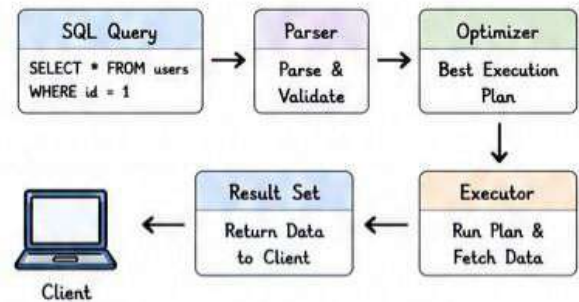
Quick Recap

A database stores structured data. SQL is used to create, read, update and delete data efficiently.

Key Concepts:

- Tables store data in rows & columns.
- Keys uniquely identify records.
- Indexes speed up data retrieval.
- Joins help to combine data from multiple tables.

SQL Query Execution Flow



Top 5 Questions in this Section

1 What is a Primary Key?



Answer (Short)

A primary key uniquely identifies each record in a table. It cannot be NULL and must be unique.

Follow-up

Can we have multiple primary keys in a table?



Example / Use in Production

user_id in users table. Used to uniquely identify each user.

id	name
1	Abhi
2	Aman



2 What is the difference between Clustered and Non-Clustered Index?



Answer (Short)

Clustered index determines the physical order of data. Non-clustered index is a separate structure pointing to data.

Follow-up

Which one is faster for search?

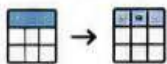


Example / Use in Production

Clustered index: On Primary Key
Non-clustered index: On email column for fast lookups.



3 What is Normalization?



Answer (Short)

Normalization is the process of organizing data to reduce redundancy and improve data integrity.

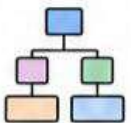
Follow-up

Name the normal forms (1NF, 2NF, 3NF, BCNF)?



Example / Use in Production

Helps to avoid data duplication and update anomalies in large databases.



4 What is the difference between INNER JOIN and LEFT JOIN?



Answer (Short)

INNER JOIN returns only matching rows.
LEFT JOIN returns all rows from left table + matches.

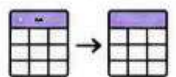
Follow-up

Which JOIN returns all rows from both tables?



Example / Use in Production

Used to combine user data with orders. LEFT JOIN helps to get users without orders.



5 What is the N+1 Query Problem?

N+1

Answer (Short)

It happens when 1 query fetches data and N additional queries fetch related data for each record.

Follow-up

How do we solve N+1 Query Problem?



Example / Use in Production

Use JOIN or fetch all data in a single query to improve performance.



Keys in SQL

- Primary Key → Uniquely identifies a row
- Foreign Key → References Primary Key
- Unique Key → Ensures all values are unique
- Composite Key → Combination of multiple columns

SQL Join Types

- INNER JOIN Only matching rows
- LEFT JOIN All left + matched rows
- RIGHT JOIN All right + matched rows
- FULL JOIN All rows from both tables

Example: Users & Orders Tables

users

id	name
1	Abhi
2	Aman
3	Neha

orders

id	user_id	amount
101	1	2500
102	1	1200
103	2	3000

SELECT u.name, o.amount

-- Returns all users



Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



4 Transactions & Query Optimization

In this section:

- ✓ ACID Properties
- ✓ Transactions
- ✓ Isolation Levels
- ✓ Locks & Deadlocks
- ✓ Query Optimization
- ✓ Execution Plan
- ✓ Best Practices



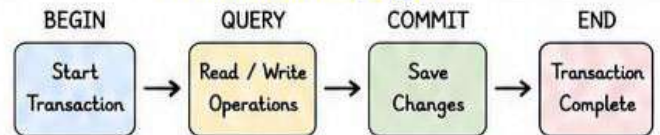
Quick Recap

Transactions ensure data consistency.
Query Optimization improves performance.

Key Concepts:

- ACID ensures reliable transactions.
- Isolation levels control visibility.
- Indexes, proper queries & execution plans make queries fast.
- Always analyze and optimize queries.

Transaction Lifecycle



ACID Properties

- A - Atomicity** : All or nothing
- C - Consistency** : Data remains valid
- I - Isolation** : Transactions don't interfere
- D - Durability** : Data persists after commit



Top 5 Questions in this Section

1	What is a Deadlock? 	Answer (Short) Deadlock occurs when two or more transactions are waiting for each other to release locks, causing a standstill.	Follow-up How can we prevent deadlocks? ?	Example / Use in Production In banking apps, transferring money in opposite directions can cause deadlocks.
2	What is Phantom Read? 	Answer (Short) Phantom read occurs when new rows are inserted by another transaction and appear in subsequent reads.	Follow-up Which isolation level prevents phantom read? ?	Example / Use in Production In reporting systems, new records inserted during report generation cause inconsistent results.
3	What is Dirty Read? 	Answer (Short) Dirty read happens when a transaction reads uncommitted data from another transaction.	Follow-up Which isolation level prevents dirty read? ?	Example / Use in Production Reading uncommitted balance in bank accounts can show incorrect data.
4	Explain Query Optimization. 	Answer (Short) Query optimization is the process of improving a query's performance by reducing execution time and resources.	Follow-up How do you optimize a slow query? ?	Example / Use in Production Optimizing slow queries in e-commerce search, reports and analytics improves user experience.
5	Why is using SELECT * considered bad? 	Answer (Short) SELECT * fetches unnecessary columns, increases I/O, memory usage and network overhead.	Follow-up When is it okay to use SELECT *? ?	Example / Use in Production It's okay in quick debugging or exploratory queries, not in production code.

Isolation Levels (from low to high isolation)

	Isolation Level	Prevents
Read Uncommitted (Dirty Read possible)	Read Uncommitted	None
Read Committed (Dirty Read prevented)	Read Committed	Dirty Read
Repeatable Read (Phantom Read possible)	Repeatable Read	Dirty Read, Non-repeatable Read
Serializable (Phantom Read prevented)	Serializable	Dirty Read, Non-repeatable Read, Phantom Read

Execution Plan Example (EXPLAIN Output)

EXPLAIN SELECT * FROM users WHERE email = 'a@b.com';							
	id	select_type	table	type	possible_keys	key	rows
	1	SIMPLE	users	ref	idx_email	idx_email	1

• id : Query execution order
• select_type : Type of SELECT
• type : Access type (ref is good)
• possible_keys : Possible indexes
• key : Index used
• rows : Estimated rows scanned

Tip: Look for 'Using index' and avoid 'Using filesort', 'Using temporary'

Query Optimization Tips

- ✓ Use proper indexes
- ✓ Avoid SELECT *
- ✓ Use LIMIT
- ✓ Avoid functions on indexed columns
- ✓ Use JOINs instead of subqueries (where possible)
- ✓ Analyze execution plans
- ✓ Keep statistics up to date





Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



5 Caching & Redis

In this section:

- ✓ Why Cache?
- ✓ Redis Basics
- ✓ Cache Strategies
- ✓ Cache Invalidation
- ✓ Data Structures
- ✓ TTL & Expiration
- ✓ Best Practices

Quick Recap

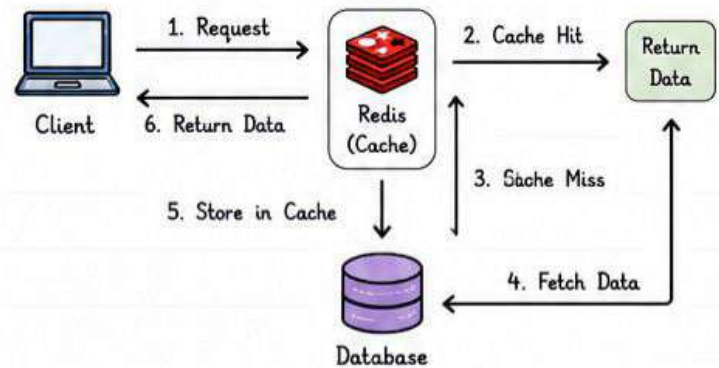
Caching stores frequently accessed data in memory for faster retrieval.

Benefits:

- Reduces database load
- Improves response time
- Increases system throughput
- Enhances scalability

Popular Cache: Redis (In-Memory Store)

Cache Aside Pattern (Most Used)



Top 5 Questions in this Section

- 1 Why do we need caching?** 

Answer (Short)
Caching reduces response time by storing frequently accessed data in memory so we don't hit the database every time.

Follow-up
Which data should we cache and why? 

Example / Use in Production
We cache product details, user sessions, leaderboards to reduce DB load and improve performance. 
- 2 What is Redis?** 

Answer (Short)
Redis is an in-memory data structure store used as a database, cache, message broker and more.

Follow-up
Why is Redis faster than MySQL? 

Example / Use in Production
Redis keeps data in memory (RAM), so read/write operations are much faster than disk-based databases. 
- 3 What happens on Cache Miss?** 

Answer (Short)
On cache miss, data is fetched from the database, returned to the client and stored in cache for future requests.

Follow-up
How can we reduce the number of cache misses? 

Example / Use in Production
We use better caching strategies, proper TTL, and pre-warming cache to reduce cache misses. 
- 4 What is Cache Invalidation?** 

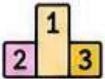
Answer (Short)
Cache invalidation is the process of deleting or updating cache when the underlying data changes to avoid stale data.

Follow-up
What are strategies for cache invalidation? 

Example / Use in Production
We use strategies like TTL, manual deletion, versioning, and event based invalidation. 
- 5 What are Redis Data Structures?** 

Answer (Short)
Redis supports String, Hash, List, Set, Sorted Set, Bitmap, HyperLogLog, Stream, Geo, etc.

Follow-up
Which data structure would you use for leaderboard? 

Example / Use in Production
We use Sorted Set (ZSET) to store scores in leaderboard with fast ranking queries. 

Common Cache Strategies

Strategy	Description
Cache Aside	App controls cache. Most used.
Read Through	Cache-aside handled by cache layer.
Write Through	Write to DB and cache at same time.
Write Behind	Write to cache, then DB asynchronously.
Refresh Ahead	Refresh cache before it expires.

Redis Data Structures Cheatsheet

Data Structure	Use Case
String	Counters, tokens, simple data
Hash	User profiles, objects
List	Queues, recent activity feeds
Set	Unique items, tags, friendships
Sorted Set	Leaderboards, rankings
Bitmap	Flags, active users, analytics
Stream	Event streaming, logs, queues

TTL (Time To Live)

TTL defines how long a key lives in Redis.

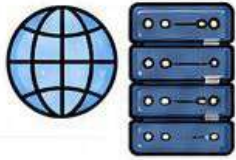
Commands:

```

SET key value EX 60 - Set key with TTL 60 sec
EXPIRE key 120 - Set TTL 120 sec
TTL key - Get remaining TTL
DEL key - Delete key
  
```

★ Best Practice: Always set appropriate TTL to avoid stale data and save memory.





Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



6 System Design & Scalability Basics

In this section:

- ✓ Scalability Concepts
- ✓ Availability & Reliability
- ✓ Load Balancer
- ✓ Reverse Proxy
- ✓ CDN (Content Delivery Network)
- ✓ Rate Limiting
- ✓ Monitoring & Logging
- ✓ Best Practices



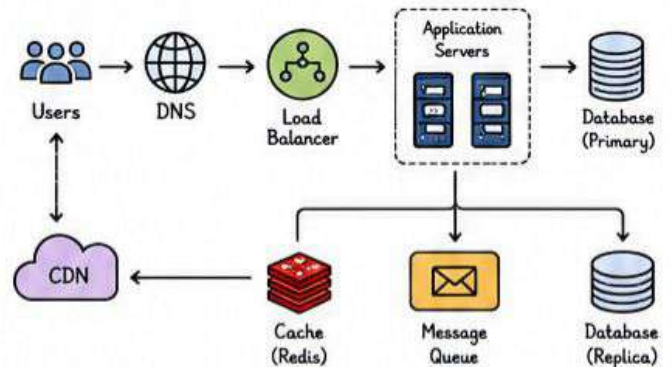
Quick Recap

System Design is about designing systems that are scalable, reliable and efficient.

Key Goals:

- High Performance
- High Availability
- Scalability
- Fault Tolerance
- Cost Efficiency

High Level Architecture of a Scalable System



Top 5 Questions in this Section

- 1** What is Scalability?



Answer (Short)
Scalability is the ability of a system to handle increasing load by adding resources to maintain performance.

Follow-up
What are the types of scalability?

Example / Use in Production
Add more servers during peak traffic (e.g., sales) to handle more users without downtime.


- 2** Horizontal vs Vertical Scalability?



Answer (Short)
Vertical means adding more power to a single server. Horizontal means adding more servers to distribute load.

Follow-up
Which one is more cost-effective?

Example / Use in Production
Horizontal scaling is more cost-effective and preferred in cloud systems.


- 3** What is a Load Balancer?



Answer (Short)
A load balancer distributes incoming traffic across multiple servers to ensure no single server is overloaded.

Follow-up
What are types of load balancers?

Example / Use in Production
Used in websites like Amazon, Netflix to distribute millions of requests across servers.


- 4** What is CDN?



Answer (Short)
CDN stores cached content in edge servers closer to users to reduce latency and speed up delivery.

Follow-up
How does CDN improve performance?

Example / Use in Production
Used to deliver static content like images, videos, JS, CSS files faster globally.


- 5** What is Rate Limiting?



Answer (Short)
Rate limiting restricts the number of requests a user can make in a given time to prevent abuse and ensure system stability.

Follow-up
How is rate limiting implemented?

Example / Use in Production
Used in APIs to prevent spam, brute force attacks, and ensure fair usage.



Scalability Approaches

Type	Description
Vertical Scaling	Add more power (CPU, RAM, Storage) to a single server. ↑
Horizontal Scaling	Add more servers and distribute the load. ↗ ↘ ↙ ↚
Auto Scaling	Automatically adds/removes servers based on traffic. ⚙️

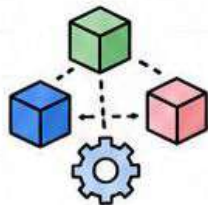
Load Balancer Algorithms

- 🔄 Round Robin - Distributes requests sequentially.
- 👤 Least Connections - Sends request to server with fewest active connections.
- # IP Hash - Routes same IP to same server.
- 🕒 Least Response Time - Sends request to server with lowest response time.

Why Do We Need These?

- ✓ Handle more users
- ✓ Improve availability
- ✓ Reduce response time
- ✓ Prevent single point of failure
- ✓ Optimize resource utilization





Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



7 Microservices & API Gateway

In this section:

- ✓ Monolith vs Microservices
- ✓ Service Decomposition
- ✓ API Gateway
- ✓ Service Discovery
- ✓ Inter-Service Communication
- ✓ Event Driven Architecture
- ✓ Circuit Breaker
- ✓ Best Practices



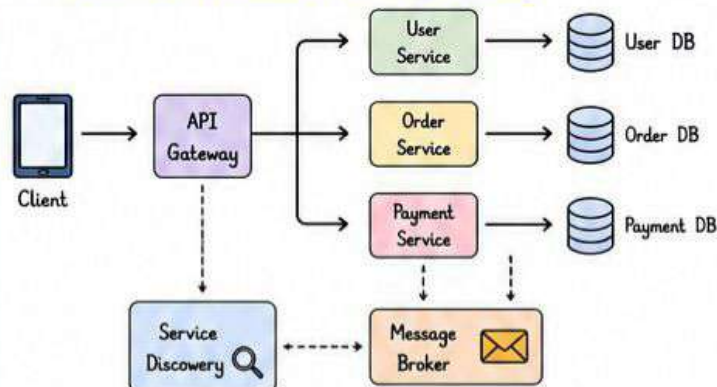
Quick Recap

Microservices break an application into small, independent services that communicate over network.

Benefits:

- Independent Deployment
- Technology Flexibility
- Scalability
- Fault Isolation
- Faster Development

Microservices Architecture with API Gateway



Top 5 Questions in this Section

- 1 What is the main difference between Monolith and Microservices?

Answer (Short)
Monolith is a single deployable unit. Microservices are small, independent services that communicate over network.

Follow-up
When should we prefer Microservices over Monolith?

Example / Use in Production
Startups use Monolith initially. Large systems like Netflix, Amazon use Microservices for scale.
- 2 What is an API Gateway?

Answer (Short)
An API Gateway is a single entry point for all client requests. It routes requests to the appropriate services.

Follow-up
What are the benefits of using an API Gateway?

Example / Use in Production
Used in systems to handle authentication, rate limiting, logging, routing, and caching.
- 3 How do microservices communicate with each other?

They communicate over network using protocols like HTTP/REST, gRPC, or messaging (Kafka, RabbitMQ).

When do we use synchronous vs asynchronous communication?

Synchronous for immediate response (HTTP, gRPC). Asynchronous for events, notifications (Kafka, MQ).
- 4 What is Service Discovery?

Service Discovery is a process where services automatically find and communicate with other services.

How does Service Discovery work in Kubernetes?

Eureka, Consul, or Kubernetes Service Discovery helps services locate each other dynamically.
- 5 What is a Circuit Breaker Pattern?

It prevents a system from continuously trying to call a failing service. It stops requests temporarily.

Which states does a circuit breaker have?

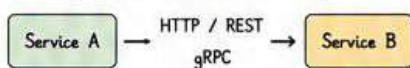
Used to avoid cascading failures. States: Closed (OK), Open (Failing), Half-Open (Testing).

Monolith vs Microservices

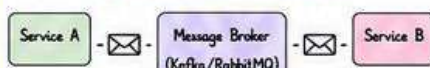
Monolith	Microservices
Single codebase	Multiple independent services
Single deployment	Independent deployment
Hard to scale	Easy to scale
Tight coupling	Loose coupling
Tech stack is same	Tech stack is different

Microservices Communication

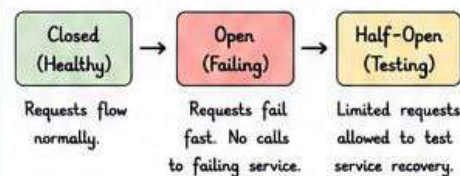
Synchronous (Immediate Response)



Asynchronous (Event Driven)



Circuit Breaker States



Tip: Circuit Breaker improves system stability by preventing repeated failures.



Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



8 SOLID Principles & Design Patterns

In this section:

- ✓ SOLID Principles
- ✓ Why SOLID Matters
- ✓ Design Patterns
- ✓ Creational Patterns
- ✓ Structural Patterns
- ✓ Behavioral Patterns
- ✓ Real-world Examples
- ✓ Best Practices



Quick Recap

SOLID principles make code more maintainable, scalable and testable. Design patterns are proven solutions to common software design problems.

Benefits:

- Easy to maintain
- Easy to extend
- Reduces bugs
- Improves readability
- Promotes reusability

SOLID Principles Overview

S	Single Responsibility Principle	A class should have only one reason to change.
O	Open/Closed Principle	Open for extension, closed for modification.
L	Liskov Substitution Principle	Objects of a superclass should be replaceable with objects of its subclass.
I	Interface Segregation Principle	Clients should not be forced to depend on interfaces they don't use.
D	Dependency Inversion Principle	Depend on abstractions, not on concrete implementations.

Top 5 Questions in this Section

1	What is the Single Responsibility Principle (SRP)? 	Answer (Short) A class should have only one responsibility or one reason to change.	Follow-up Can you give an example of violating SRP?	Example / Use in Production Separating 'UserService' and 'EmailService' makes code easier to maintain and test.	
2	What is the Open/Closed Principle (OCP)? 	Software entities (classes, modules, functions) should be open for extension but closed for modification.	How do we achieve OCP in real projects?	Using interfaces and abstract classes to add new behavior without modifying existing code.	
3	What is Liskov Substitution Principle (LSP)? 	Subtypes must be substitutable for their base types without affecting the correctness of the program.	Can you give an example of LSP violation?	If 'Square' extends 'Rectangle' and breaks the behavior of 'Rectangle', LSP is violated.	
4	What is Interface Segregation Principle (ISP)? 	Clients should not be forced to depend on interfaces they don't use.	Why is ISP important in large systems?	Splitting a large interface into smaller, client-specific interfaces reduces dependency.	
5	What is Dependency Inversion Principle (DIP)? 	High level modules should not depend on low level modules. Both should depend on abstractions.	How does DIP help in testing?	Using interfaces allows easy mocking of dependencies and improves unit testing.	

Common Design Patterns

Creational	Singleton, Factory, Abstract Factory, Builder, Prototype
Structural	Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight
Behavioral	Strategy, Observer, Command, Template Method, State, Iterator, Chain of Responsibility, Mediator, Memento

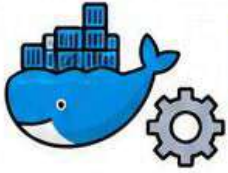
Example: Singleton Pattern

```
class Singleton {
    private static Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```



When to Apply Which Pattern?

- ★ Singleton: When only one instance is needed (e.g., DB connection, Logger)
- ⚙ Factory: When object creation logic is complex or needs flexibility
- 🛡 Strategy: When you have multiple algorithms and want to switch between them
- 👁 Observer: When one-to-many dependency exists between objects
- 🏠 Facade: When you want a simple interface to a complex subsystem



Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



9 Docker & CI/CD Basics

In this section:

- ✓ Docker Fundamentals
- ✓ Images vs Containers
- ✓ Dockerfile Basics
- ✓ Docker Compose
- ✓ Multi-stage Build
- ✓ CI/CD Pipeline
- ✓ GitHub Actions
- ✓ Deployment Strategies
- ✓ Best Practices



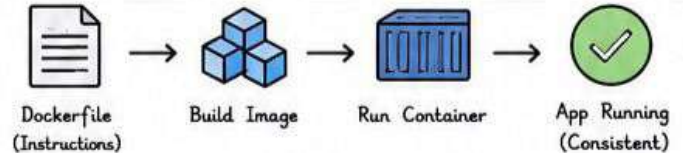
Quick Recap

Docker helps us package applications and their dependencies into containers so they can run consistently anywhere.

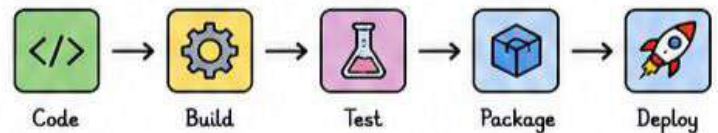
Benefits:

- Consistent environment
- Lightweight & portable
- Faster deployment
- Better resource utilization
- Easy scaling

How Docker Works?



CI/CD Pipeline Overview



Top 5 Questions in this Section

- | | | | |
|---|---|--|---|
| 1 What is the difference between Docker Image and Container?
 | Answer (Short)
An Image is a blueprint (read-only template). A Container is a running instance of an image. | Follow-up
Can a container be created without an image?
 | Example / Use in Production
We create one image of our app and run multiple containers of it for different environments.
 |
| 2 What is the use of Dockerfile?
 | Answer (Short)
Dockerfile contains instructions to build a Docker Image in a consistent and automated way. | Follow-up
Why is it better than building images manually?
 | Example / Use in Production
Used to define base image, dependencies, copy files, expose ports, and run commands.
 |
| 3 What is Docker Compose?
 | Answer (Short)
Docker Compose is a tool for defining and running multi-container Docker applications using a YAML file. | Follow-up
When would you use Compose over run multiple containers?
 | Example / Use in Production
Used to run multi-service apps like backend, database, redis, nginx together.
 |
| 4 What is CI/CD?
 | Answer (Short)
CI/CD stands for Continuous Integration / Continuous Delivery or Deployment. | Follow-up
How does CI/CD improve software development?
 | Example / Use in Production
Automates build, test, and deployment process to deliver code faster and with fewer errors.
 |
| 5 What is a Multi-stage Docker Build?
 | Answer (Short)
A technique to use multiple FROM statements in Dockerfile to reduce final image size. | Follow-up
What are the benefits of multi-stage builds?
 | Example / Use in Production
Used to build in one stage and copy only necessary files to final stage → smaller, secure images.
 |

Dockerfile Basics

```

FROM node:18-alpine      # Base Image
WORKDIR /app             # Set Workdir
COPY package*.json ./    # Copy files
RUN npm install          # Install Deps
COPY . .                 # Copy Source
EXPOSE 3000              # Expose Port
CMD ["node", "server.js"] # Start App
  
```



CI/CD Pipeline Example (GitHub Actions)

```

name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '18'
      - run: npm install
      - run: npm test
      - run: docker build -t myapp .
      - run: docker push myapp:latest
  
```



Deployment Strategies

- Blue-Green Deployment**
 Two identical environments (Blue & Green). Switch traffic.
- Rolling Deployment**
 Update instances one by one.
- Canary Deployment**
 Deploy to small % of users first.
- Recreate Deployment**
 Stop old version, then start new.



Backend Interview Handbook

Top 50 Backend Interview Questions & Answers

Your Complete Guide to Backend Interviews (SDE-1 / SDE-2)



10 Monitoring, Logging & Observability

In this section:

- ✓ Why Monitoring Matters
- ✓ Logs vs Metrics vs Traces
- ✓ Monitoring Tools
- ✓ Centralized Logging
- ✓ Alerting & Notifications
- ✓ Dashboards & Visualization
- ✓ Distributed Tracing
- ✓ Best Practices



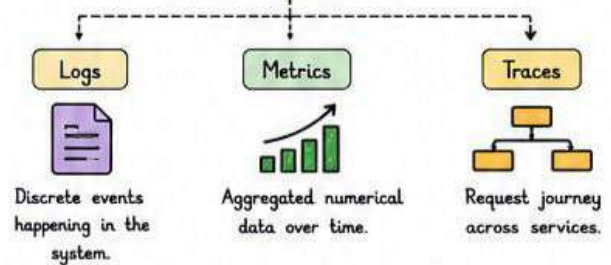
Quick Recap

Monitoring and logging help us understand system health, debug issues faster, and improve user experience.

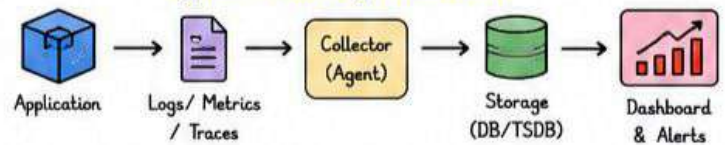
Benefits:

- Detect issues early
- Improve system reliability
- Faster root cause analysis
- Better performance insights
- Data-driven decisions

Observability Pillars



Typical Monitoring Architecture



Top 5 Questions in this Section

- 1** Why is monitoring important?



Answer (Short)
It helps detect issues early, understand system behavior, and ensure high availability and performance.

Follow-up
What are the consequences of not having proper monitoring?

Example / Use in Production
Without monitoring, outages may go unnoticed, causing poor user experience and revenue loss.


- 2** What's the difference between Logs, Metrics & Traces?



Answer (Short)
Logs are events, Metrics are aggregated numbers, Traces show request flow across services.

Follow-up
When would you use traces instead of logs?

Example / Use in Production
Use traces to debug slow requests across multiple microservices where logs are hard to correlate.


- 3** What tools do you use for monitoring?



Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana), Datadog, New Relic, Jaeger.

Which tool would you choose for metrics collection and why?

Prometheus for metrics collection (its pull-based, highly scalable, and has great alerting).


- 4** What is centralized logging?



Centralized logging collects logs from all services in one place for easier search, analysis, and alerting.

How do you implement centralized logging in a microservices system?

Use ELK Stack or Loki + Grafana. Each service sends logs to a central log aggregator.


- 5** What is distributed tracing?



Distributed tracing tracks a request as it flows through multiple services to identify latency and failures.

Name a tracing tool and how it works?

Jaeger / Zipkin - Each service adds trace IDs to requests. Helps visualize the complete flow.



JAEGER

Logs vs Metrics vs Traces

Type	Purpose	Example
	Record discrete events	"User login successful"
	Measure system performance	CPU usage = 70%
	Follow request flow end-to-end	Request took 2s across 5 services

Common Monitoring Tools

-  Prometheus - Metrics collection & alerting
-  Grafana - Visualization & dashboards
-  ELK Stack - Centralized logging & analysis
-  Jaeger - Distributed tracing
-  Alertmanager - Alert routing & management

Best Practices

- ✓ Monitor RED: Rate, Errors, Duration
- ✓ Set meaningful alerts (avoid alert fatigue)
- ✓ Use dashboards for key system metrics
- ✓ Correlate logs, metrics & traces
- ✓ Test your alerts and runbooks regularly

